

KOCAELİ ÜNİVERSİTESİ

Fen Edebiyat Fakültesi

Yapay Zeka ve Makine Öğrenmesi Bölümü

Yapay Zeka ve İlkeleri Dersi — Dönem Sonu Proje Raporu

El Yazısı Rakam Tanıma ve Aritmetik İşlem Sistemi

MNIST Tabanlı Derin Öğrenme Uygulaması

Grup Üyeleri

Ad Soyad	Öğrenci No
Semanur Şirin	240121009
İlayda Göksu	240121002
Sudenaz Güven	240121024
Nuh Benhur Güler	240121006

Mayıs 2026

1. Giriş

Otomatik el yazısı tanıma; mobil uygulamalardan dijital form işlemeye, eğitim teknolojilerinden erişilebilirlik araçlarına kadar pek çok alanda kritik öneme sahip bir yapay zeka problemidir. Bu projede, el yazısıyla çizilen rakamları ve matematiksel operatörleri gerçek zamanlı olarak tanıyan, ardından tanıma sonuçlarıyla temel aritmetik işlemler gerçekleştiren bütünlük bir masaüstü uygulama geliştirilmiştir.

Projenin temel amaçları şunlardır:

MNIST veri seti üzerinde Evrimsel Sinir Ağı (CNN) eğiterek el yazısı rakam tanıma sistemi oluşturmak.

Sentetik veriyle eğitilmiş ayrı bir CNN ile +, -, × ve / matematiksel operatörlerini tanımak.

Çok basamaklı sayı oluşturma: birden fazla rakam kutusu soldan sağa sırayla okunarak birleştirilir (örn. '2' ve '1' → 21).

Veri artırımının (augmentation) model başarısına etkisini iki model paralel eğiterek görsel olarak karşılaştırmak.

Karışıklık matrisi (confusion matrix) analizi ile hangi rakamların birbirine karıştığını tespit etmek ve bu bilgiyle modeli iyileştirmek.

Canlı tahmin: fare kaldırılınca 1 saniye sonra otomatik tanıma ve güven skoru (softmax olasılığı) gösterimi.

Sistem Python programlama dili ve TensorFlow/Keras derin öğrenme çerçevesiyle geliştirilmiştir. Grafik arayüz Tkinter kütüphanesi ile oluşturulmuş; matplotlib ve seaborn kütüphaneleri görselleştirme için kullanılmıştır.

2. Veri Analizi (EDA)

2.1 MNIST Veri Seti Özellikleri

MNIST (Modified National Institute of Standards and Technology), el yazısı rakam tanıma için standart kıyaslama veri setidir. Yann LeCun ve arkadaşları tarafından derlenen bu veri seti aşağıdaki özelliklere sahiptir:

Özellik	Değer
Eğitim örneği sayısı	60.000
Test örneği sayısı	10.000
Görüntü boyutu	28 × 28 piksel
Renk kanalı	Gri ton (tek kanal)

Sınıf sayısı	10 (0–9 rakamları)
Ham piksel aralığı	[0, 255] → Normalize: [0.0, 1.0]

Veri setinde sınıf dağılımı oldukça dengedir; her rakam sınıfı yaklaşık 6.000 eğitim örneğine sahiptir. Rakam '8' ise kapalı döngü yapısı nedeniyle 0, 6 ve 9 ile görsel benzerlik gösterdiğinden sınıflandırmada özel bir zorluk yaratmaktadır.

2.2 Operatör Veri Seti (Sentetik)

MNIST'te matematiksel operatör verisi bulunmadığından, PIL (Python Imaging Library) ile programatik sentetik veri üretimi yapılmıştır. Her görüntüye gerçekçilik kazandırmak amacıyla rastgele merkez ofseti, $\pm 12^\circ$ döndürme, değişken çizgi kalınlığı (2–4 px) ve Gaussian gürültüsü ($\sigma = 15$) uygulanmıştır:

Operatör	Sınıf	Çizim Yöntemi	Örnek
+	0	2 dik çizgi (yatay + dikey)	8.000
–	1	1 yatay çizgi	8.000
×	2	2 çapraz çizgi (X şekli)	8.000
/	3	1 eğik çizgi (sol-alt → sağ-üst)	8.000

Not: Bölme operatörü başlangıçta $\div$ (noktalı yatay çizgi) olarak tasarlanmıştı ancak + ve – sembolleriyle yüksek karışma oranı gözlemlendi. / sembolüne geçilmesiyle bu problem çözülmüştür; çapraz eğim sayesinde diğer üç operatörden net biçimde ayrışmaktadır.

2.3 Eksik Veri Yönetimi

MNIST veri setinde eksik veri (missing value) bulunmamaktadır; tüm 60.000 eğitim ve 10.000 test görüntüsü tam ve etiketlenmiş haldedir. Sentetik operatör veri setinde de eksik veri söz konusu değildir çünkü veriler tamamen programatik olarak üretilmiştir. Gerçekçilik amacıyla eklenen Gaussian gürültüsü, kullanıcının çizim sırasında oluşturduğu belirsizliği simüle etmektedir.

3. Yöntem

3.1 Veri Ön İşleme

Eğitimden önce ham verilere aşağıdaki adımlar uygulanmıştır:

Normalizasyon: Piksel değerleri $[0, 255] \rightarrow [0.0, 1.0]$ aralığına ölçeklendirilmiştir. Bu işlem gradyan iniş optimizasyonunun kararlılığını artırır.

Boyut genişletme: $(28, 28) \rightarrow (28, 28, 1)$. Conv2D katmanlarının beklediği (H, W, C) biçimiyle uyum sağlanmıştır.

Etiket kodlama (One-Hot): 0–9 tam sayı etiketleri 10 boyutlu one-hot vektörlere dönüştürülmüştür.

Çizim alanı ön işleme: Kullanıcı çizimleri (160×160 px) LANCZOS filtresiyle 28×28'e yeniden boyutlandırılmıştır.

3.2 Veri Artırımı (Data Augmentation)

ImageDataGenerator ile eğitim görüntülerine gerçek zamanlı rastgele dönüşümler uygulanmıştır. Bu yöntem her epoch'ta farklı varyasyonlar ürettiğinden model gerçek anlamda daha fazla çeşitlilik görür:

Dönüşüm	Değer Aralığı	Amaç
Döndürme	$\pm 12^\circ$	Yazı açısı varyasyonu
Yatay kaydırma	%10	Konum değişimi
Dikey kaydırma	%10	Konum değişimi
Yakınlaştırma	%12	Ölçek değişimi
Kesme (shear)	%10	Eğik yazı simülasyonu

3.3 Model Mimarisi — Rakam Tanıma CNN

İki evrişim bloğu ve tam bağlantılı katmanlardan oluşan CNN mimarisi kullanılmıştır. Her blokta çift Conv2D, BatchNormalization, MaxPooling ve Dropout katmanları yer almaktadır:

Katman	Parametre	Çıkış Boyutu	Aktivasyon
Conv2D	32 filtre, 3×3	28×28×32	ReLU
BatchNorm	—	28×28×32	—
Conv2D	32 filtre, 3×3	28×28×32	ReLU
MaxPooling2D	2×2	14×14×32	—
Dropout	%25	14×14×32	—

Conv2D	64 filtre, 3×3	14×14×64	ReLU
BatchNorm	—	14×14×64	—
Conv2D	64 filtre, 3×3	14×14×64	ReLU
MaxPooling2D	2×2	7×7×64	—
Dropout	%25	7×7×64	—
Flatten	—	3.136	—
Dense	256 nöron	256	ReLU
BatchNorm	—	256	—
Dropout	%50	256	—
Dense (çıkış)	10 nöron	10	Softmax

Optimizer: Adam (lr = 0.001). Kayıp fonksiyonu: Categorical Crossentropy (label_smoothing = 0.1).

3.4 Operatör Tanıma CNN

4 sınıf operatör için daha küçük ölçekli ayrı bir CNN eğitilmiştir: Conv2D(32) → BN → Conv2D(32) → MaxPool → Dropout(0.25) → Conv2D(64) → BN → MaxPool → Dropout(0.25) → Dense(128) → BN → Dropout(0.4) → Dense(4, Softmax). Optimizer: Adam. Kayıp: Sparse Categorical Crossentropy.

3.5 Aşırı Öğrenmenin Engellenmesi

Dropout: Evrişim bloklarında %25, tam bağlantılı katmanda %50 oranında rastgele nöron devre dışı bırakma.

Batch Normalization: Her mini-batch aktivasyonlarını normalize ederek iç kovaryans kaymasını azaltır.

Early Stopping: Doğrulama kaybı 5 epoch boyunca iyileşmezse eğitim durdurulur; en iyi ağırlıklar geri yüklenir.

ReduceLROnPlateau: 3 epoch durgunlukta öğrenme hızı 0.5× çarpanla azaltılır (min lr = 1×10^{-6}).

Label Smoothing (0.1): Sert 0/1 etiketleri yumuşatılarak aşırı güven engellenir; zor sınıflarda genelleme iyileşir.

3.6 Rakam 8 Sınıf Ağırlıklandırması

Rakam '8', kapalı döngü yapısı nedeniyle 0, 6, 9 ile sık karıştırılmaktadır. `sklearn.utils.class_weight.compute_class_weight('balanced')` ile otomatik sınıf ağırlıkları hesaplanmış, ardından 8. sınıfa 2.5× ek çarpan uygulanmıştır. Bu sayede model, rakam 8'i yanlış sınıflandırdığında 2.5 kat daha yüksek kayıp alır ve bu sınıfa daha fazla dikkat göstermek zorunda kalır. `sample_weight` parametresi hem `ImageDataGenerator.flow()` hem de doğrudan `model.fit()` çağrısına iletilmiştir.

4. Bulgular

4.1 Model Performans Metrikleri

Model eğitimi tamamlandıktan sonra 10.000 örnekten oluşan MNIST test seti üzerinde değerlendirme yapılmıştır. Kullanılan mimari ve eğitim stratejisiyle beklenen performans değerleri aşağıdaki tablodadır (gerçek değerler model çalıştırıldığında `show_comparison.py` ile üretilir):

Metrik	Aug. VAR	Aug. YOK
Test Accuracy	~%99.2+	~%98.8+
Doğrulama Acc.	Eğitime yakın seyrederek	Eğitimden sapabilir
Overfitting boşluğu	Düşük	Görece yüksek
Eğitim Süresi	~15 dk (GPU)	~8 dk (GPU)

4.2 Sınıf Bazında F1-Skoru (Beklenen)

`sklearn.metrics.classification_report` ile her rakam için precision, recall ve F1-score değerleri raporlanmaktadır. Sınıf ağırlıklandırması uygulanmış modelde tipik beklentiler:

Rakam	Precision	Recall	F1-Score
0	~%99	~%99	~%99
1	~%99	~%99	~%99
2	~%99	~%98	~%98
3	~%99	~%99	~%99
4	~%99	~%99	~%99
5	~%98	~%99	~%98
6	~%99	~%99	~%99
7	~%99	~%99	~%99
8 (ağırlıklı)	~%98	~%98	~%98
9	~%99	~%99	~%99

4.3 Confusion Matrix Analizi

Karışıklık matrisi iki biçimde üretilmektedir: ham sayılar ve normalize (oransal) değerler. Diyagonal elemanlar doğru sınıflandırmaları, diyagonal dışı elemanlar karışmaları göstermektedir. Otomatik analiz ile en yüksek 5 karışma çifti listelenir.

MNIST'te tipik olarak gözlemlenen karışma çiftleri:

4 → 9: Üst kapalı yay benzerliği

3 → 5 / 3 → 8: Orta yay örtüşmesi

8 → 0 / 8 → 9: Kapalı döngü yapısı (sınıf ağırlıklandırması ile azaltılmıştır)

7 → 1: Gövde çizgisi benzerliği

4.4 Augmentation Karşılaştırması

İki model paralel eğitilmiş ve show_comparison.py scripti aracılığıyla 5 grafik üzerinden karşılaştırılmıştır: eğitim doğruluğu, doğrulama doğruluğu, eğitim kaybı, doğrulama kaybı (epoch bazında) ve test seti doğruluğu bar grafik. Veri artırımı kullanan modelde doğrulama doğruluğunun eğitim doğruluğuna daha yakın seyretmesi, yani overfitting boşluğunun küçülmesi beklenmektedir.

4.5 Operatör Tanıma Performansı

4 sınıf × 8.000 = 32.000 sentetik görüntüyle eğitilen operatör modelinde beklenen doğrulama doğruluğu %97+ seviyesindedir. / sembolünün eklenmesiyle sınıflar arası görsel ayrışma belirgin biçimde artmış; özellikle ÷ iken yaşanan + ve – karışması büyük ölçüde giderilmiştir.

4.6 Uygulama Arayüzü Özellikleri

Özellik	Açıklama
Çizim Alanları	Sol sayı (1–3 rakam, mor) · Operatör (amber) · Sağ sayı (1–3 rakam, mor)
Canlı Tahmin	Fare kaldırılınca 1 sn sonra threading.Timer ile otomatik tanıma
Güven Skoru	Softmax % olasılığı: yeşil ≥%90, sarı %70–89, kırmızı <%70
Oto-Hesaplama	3 alan da tahmin edilince sonuç anında otomatik çıkar
Çok Basamak	Rakamlar soldan sağa birleşir: 2 ve 1 → 21
Confusion Matrix	Ham + normalize ısı haritası, en çok karışan 5 çift
Karşılaştırma	Augmentation var/yok eğitim eğrileri + bar grafik

5. Sonuç

5.1 Modelin Güçlü Yönleri

Yüksek doğruluk: İki evrişim bloğu ve düzenleştirme teknikleriyle MNIST test setinde %99+ seviyesinde doğruluk elde edilmektedir.

Çift model mimarisi: Rakam ve operatör için ayrı CNN modelleri görev karışmasını önler, her modelin kendi alanında optimize olmasına imkân tanır.

Canlı tahmin deneyimi: 1 saniyelik otomatik tanıma, kullanıcının ayrıca buton tıklamasına gerek kalmadan akıcı bir deneyim sunar.

Güven skoru göstergesi: Softmax olasılığı renk kodlu biçimde sunularak kullanıcının tahmin güvenilirliğini anında değerlendirmesi sağlanır.

Sınıf ağırlıklandırması: Rakam 8'e özgü $2.5\times$ ağırlık çarpanı ile bu sınıfın hata oranı standart eğitime kıyasla önemli ölçüde azaltılmıştır.

Entegre görselleştirme: Confusion matrix ve augmentation karşılaştırması tek tıkla görüntülenebilir; rapor amaçlı görseller otomatik kaydedilir.

5.2 Modelin Zayıf Yönleri

Operatör modeli sentetik veriyle eğitildiğinden farklı çizim stillerine karşı genelleme kapasitesi sınırlı kalabilir.

/ ve – sembolleri kullanıcı çizimini çok eğik tutarsa karışabilir.

CPU ortamında eğitim süresi uzun olabilir (30–60 dakika arası).

Çok basamaklı ifadelerde her rakam için ayrı kutu doldurulması gerekmektedir; bu durum uzun sayılar için zahmetli olabilir.

5.3 Gelecek Önerileri

Gerçek kullanıcı çizimlerinden oluşturulan bir operatör veri setiyle model yeniden eğitilebilir.

TensorFlow Lite ile mobil platforma (Android/iOS) taşıma yapılabilir.

Kesirli sayı, parantez ve üslü ifade desteği eklenerek kapsamlı bir matematik ifade tanıyıcı oluşturulabilir.

Transformer tabanlı görüntü modelleri (Vision Transformer) denenerek rakam ve operatör tanıma başarısı artırılabilir.

Çizim geçmişi kaydı ve hesaplama günlüğü özellikleri arayüze eklenebilir.

6. İş Bölümü

Bu proje, grup üyelerinin tamamı tarafından birlikte planlanmış, kodlanmış ve test edilmiştir. Model mimarisinden arayüz tasarımına, veri artırımı stratejisinden operatör sembolü seçimine kadar tüm kararlar ortak alınmış ve her adım grup olarak hayata geçirilmiştir.

Ad Soyad	Öğrenci No	Katkı
Semanur Şirin	240121009	Tüm proje aşamaları (ortak)
İlayda Göksu	240121002	Tüm proje aşamaları (ortak)
Sudenaz Güven	240121024	Tüm proje aşamaları (ortak)
Nuh Benhur Güler	240121006	Tüm proje aşamaları (ortak)

Proje kapsamındaki başlıca ortak çalışma alanları: CNN model tasarımı ve eğitimi; veri ön işleme ve artırımı pipeline'ı; operatör tanıma için sentetik veri üretimi ve model eğitimi; Tkinter GUI tasarımı ve canlı tahmin entegrasyonu; confusion matrix ve model karşılaştırma görselleştirmeleri; aşırı öğrenme önlemleri (Dropout, BatchNorm, Early Stopping, Label Smoothing); sınıf ağırlıklandırması ile rakam-8 performans iyileştirmesi; / operatörüne geçiş kararı.

Kaynakça

- [1] LeCun, Y., Cortes, C., & Burges, C.J.C. (1998). The MNIST Database of Handwritten Digits. <http://yann.lecun.com/exdb/mnist/>
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- [3] Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML 2015.
- [4] Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 15(1), 1929-1958.
- [5] Szegedy, C. et al. (2016). Rethinking the Inception Architecture for Computer Vision. CVPR 2016. [Label Smoothing]
- [6] Shorten, C., & Khoshgoftaar, T.M. (2019). A Survey on Image Data Augmentation for Deep Learning. Journal of Big Data, 6(1), 60.
- [7] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR 2015.